Welcome to TIPS

Technical Interview Preperation System

Username

Enter your username

Password

Enter your password

Login

Create Account

Hello, JohnGamecock! 👋

Ready to sharpen your coding skills today?

5 Days

Current Streak - Keep it up!

View Questions

Browse and solve coding problems from various courses

Explore Questions

Daily Challenge

Take on today's challenge and maintain your streak

Start Challenge

Your Progress

Problems Solved
24

Courses Active
3

Achievements
7

Daily Challenge

Friday, April 3, 2026

Maximum Subarray

Medium C++ • CSCE 350

Problem Description

Given an integer array nums, find the subarray with the largest sum, and return its sum.

Example 1:
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.

Example 2:
Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.

Hint

Try Kadane's algorithm. Keep track of the maximum sum ending at the current position and the overall maximum sum.

Test Cases

Input: [-2,1,-3,4,-1,2,1,-5,4]
Expected Output: 6

Input: [1]
Expected Output: 1

Start Solving

Your Streak

5
Days

Complete today's challenge to increase your streak!

Challenge Stats

Completed
12 / 30

Success Rate
92%

Leaderboard

1.	EmmaWilson	28
2.	JohnGamecock	24
3.	AlexTech	21
4.	SarahCoder	18

Questions

Search questions...

All Courses

All Difficulties

All Languages

Two Sum Easy

Language: Java • Course: CSCE 146

Solve

Valid Parentheses Easy

Language: Java • Course: CSCE 146

Solve

Merge Two Sorted Lists Easy

Language: Java • Course: CSCE 146

Solve

Binary Search Tree Validation Medium

Language: C++ • Course: CSCE 350

Solve

Longest Palindromic Substring Medium

Language: C++ • Course: CSCE 350

Solve

Graph Cycle Detection Hard

Language: C++ • Course: CSCE 350

Solve

Back to Questions

Valid Parentheses

Easy Java • CSCE 146

Problem Description

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:
1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:
Input: s = "()"
Output: true

Example 2:
Input: s = "()[]{}"
Output: true

Example 3:
Input: s = "(]"
Output: false

Hint

Test Cases

Input: "()"

Expected Output: true

Input: "()[]{}"

Expected Output: true

Input: "(]"

Expected Output: false

Write Code
View Solution
Discussion

Problem Stats

Acceptance Rate	68.5%
Total Submissions	1,247
Total Accepted	854

Back to Problem

Valid Parentheses

Easy

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:
1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:
Input: s = "()"
Output: true

Example 2:
Input: s = "()[]{}"
Output: true

Example 3:
Input: s = "(]"
Output: false

Test Cases

Input: "()"
Output: true

Input: "()[]{}"
Output: true

Input: "(]"
Output: false

Write Code

Java

Write your code here...

Submit

Back to Problem

Valid Parentheses - Solution

Easy Java

Solution Code

```
public boolean isValid(String s) {
    // Create a stack to store opening brackets
    Stack<Character> stack = new Stack<>();

    // Iterate through each character
    for (char c : s.toCharArray()) {
        // If it's an opening bracket, push to stack
        if (c == '(' || c == '{' || c == '[') {
            stack.push(c);
        }
        // If it's a closing bracket
        else {
            // Check if stack is empty (no matching opening bracket)
            if (stack.isEmpty()) return false;

            // Pop the top and check if it matches
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == '}' && top != '{') return false;
            if (c == ']' && top != '[') return false;
        }
    }

    // Valid only if stack is empty (all brackets matched)
    return stack.isEmpty();
}
```

Explanation

Approach: Stack

A stack is perfect for this problem because of its LIFO (Last In, First Out) property, which matches how brackets must be closed.

Algorithm:
1. Create an empty stack
2. For each character:
   - If opening bracket: push to stack
   - If closing bracket: pop from stack and verify it matches
3. After processing all characters, stack should be empty

Time Complexity: O(n) - Single pass through string
Space Complexity: O(n) - Stack can hold up to n/2 opening brackets

Why it works: Each closing bracket must match the most recent unmatched opening bracket, which is exactly what a stack tracks.

Learning Tips

- Try to understand the approach before memorizing the code
- Practice implementing the solution yourself without looking
- Consider edge cases and test your understanding
- Think about how you could explain this solution to someone else

Actions

View Problem
Try It Yourself
Join Discussion

Problem Details

Difficulty
Easy

Language
Java

Course
CSCE 146

Practice Makes Perfect

Don't just read the solution - implement it yourself to truly understand the concept!